📘 MODULE 10: FIELD EXTRACTION

10.1 Concept

Raw events are text. **Fields must be extracted** before you can run stats count by user. Extraction can happen at **search time** (most fields) or **index time** (rare).

10.2 Types

Type	When	Notes
Automatic (search-time)	While you search	Splunk auto-finds key=value pairs, and add-ons define fields per sourcetype
Search-time with rex	In your search	Flexible, temporary, per search
Field Extractor (UI)	Settings/Event → Extract Fields	Creates a permanent extraction in props.conf
Indexed (index-time)	Written when data is indexed	Fast to search with tstats, but costs storage and can't be changed afterward. Used sparingly.

Default indexed fields: host, source, sourcetype, index, _time.

Search-time is preferred. You can fix or change it anytime, without re-indexing.

10.3 Regex essentials (only what you need)

Pattern	Matches
\d	A digit
\d+	One or more digits
\w / \w+	Word character(s): letters, digits, _
\S+	One or more **non-space** characters
\s	Whitespace
.	Any character
.*	Anything, greedy
.*?	Anything, **lazy** (shortest match)

Pattern	Matches
[^]+	Anything except a space
(…)	**Capture group** (the part you extract)
(?<name> …)	**Named** capture group, which becomes the field name
(?: …)	Group without capturing
?	Optional
^ / $	Start / end of the string
\.	A literal dot (. alone means any character)

Capture group concept: parentheses mark the part to keep. Everything outside the parentheses is only a landmark that locates it.

10.4 rex

Syntax

spl

| rex [field=<field>] "<regex with (?<name>…)>"

Default field is _raw.

Query 1: extract IP and user from failed logins

Sample event:
Oct 11 10:15:01 web1 sshd[812]: Failed password for invalid user admin from 192.168.1.5 port 22 ssh2

spl

index=* sourcetype=linux_secure "Failed password"

| rex "Failed password for (?:invalid user)?(?<user>\S+) from (?<src_ip>\d{1,3}(?:\.\d{1,3}){3})"

| stats count by user, src_ip

| sort - count

What it does: pulls user and src_ip out of the text and counts failures per pair.
Line-by-line:

- Failed password for is a literal landmark.

- (?:invalid user)? is an optional, non-captured group, because this phrase only appears sometimes.
- (?<user>\S+) captures the next run of non-space characters into user.
- from is a landmark.
- (?<src_ip>\d{1,3}(?:\.\d{1,3}){3}) captures four number groups separated by dots.

Expected result: a table such as admin | 192.168.1.5 | 37.
Common mistakes:

- Forgetting the ?<name>, which means no field is created.
- Unescaped dots.
- Greedy .* swallowing too much.
- Regex doesn't match a few events because of format variations, so check with | where isnull(user).

Interview explanation: "I use rex with named capture groups to extract fields at search time when no extraction exists. For a permanent fix, I define it in props.conf or with the Field Extractor."

Query 2: multiple matches

spl

| rex max_match=0 "(?<ips>\d{1,3}(?:\.\d{1,3}){3})"

max_match=0 extracts **all** matches in the event into a multivalue field. The default is 1.

Query 3: key=value text with rex

Event: action=blocked src=10.0.0.9 dst=8.8.8.8

spl

| rex "src=(?<src>\S+) dst=(?<dst>\S+)"

Query 4: replace/mask text (sed mode)

spl

| rex field=_raw mode=sed "s/password=\S+/password=****/g"

This only changes the **search-time view**. It does not alter indexed data. For permanent masking, use props/transforms with SEDCMD (Module 23).

10.5 regex command (filter, not extract)

spl

index=main | regex _raw="Failed password for (invalid user)?root"

regex **keeps or removes events** based on a pattern. It does not create fields. Use != to exclude: | regex _raw!="debug".

10.6 extract / kv (key=value and delimiters)

spl

| extract pairdelim=" " kvdelim="="

- pairdelim: the separator between pairs. kvdelim: the separator between key and value.
- Splunk already auto-extracts simple key=value data, so this is for unusual delimiters, e.g. pairdelim=";" kvdelim=":".
- kv is an alias of extract.
- For CSV-like lines, split in eval: | eval parts=split(_raw,","), user=mvindex(parts,2).

10.7 The Field Extractor (UI)

1. Search for events, click an event → **Event Actions → Extract Fields**.
2. Choose **Regular Expression** (or Delimiters).
3. Highlight example values, name the field, **Next → Validate → Save**.
4. Splunk writes it into props.conf as an EXTRACT- setting in the app you choose.
5. Set permission to **App** or **All apps** to share it, otherwise only you see it.

10.8 Troubleshooting: "regex not extracting"

Check	Fix
Missing ?<name>	Add a named group
Test on a few events first	Use | head 20 | rex … and | table _raw field
Wrong field being searched	Add field= or confirm _raw has the text
Special chars	Escape . ( ) [ ] | ? * with \
Field extracted in my search but not elsewhere	Permission set to Private
Regex works on one event but not others	Variants of the log format, so make parts optional with ?

10.9 Hands-on

1. Take 10 splunkd events and extract component using rex from the raw text (the word after the log level).
2. Using your auth.log data, extract user and src_ip for failed logins and list the top 5 IPs.
3. Write a regex command to keep only events containing “Accepted password”.
4. Use makeresults with eval _raw="action=blocked src=10.0.0.9 dst=8.8.8.8" and extract src and dst.

10.10 Interview Q&A

- **Search-time vs index-time extraction?** Search-time happens during searching (flexible, preferred). Index-time is written at ingest (fixed, costs storage).
- **rex vs regex?** rex extracts fields, regex filters events.
- **What is a capture group?** The part of a regex in parentheses that is extracted. A named group becomes the field name.
- **How do you make an extraction permanent?** Field Extractor or props.conf (EXTRACT- setting), with sharing permissions.
- **What does max_match=0 do?** Extracts all matches into a multivalue field.

10.11 Revision

Search-time preferred · rex "(?<field>regex)" · \S+ non-space · escape dots · regex filters, rex extracts · test with head 20 · permanent = Field Extractor/props.conf.